EDX Incubator Phase 2 Form

Completed by Rowan Terra and Paige DiGirolamo

Section 1: Basic Information

1. Project Name

DiGiTerra

2. Primary Technical Point of Contact (Name & Email)

Rowan Terra, terrar@duq.edu, rowan.terra@netl.doe.gov Paige DiGirolamo, paige@digirolamo.me

---

Section 2: User Account Provisioning

3. User Access Requirements

If deploying only as a desktop application (compiled with PyInstaller), GCP access may not be required. GCP access would only be needed if deploying the containerized web version. rowan.terra@netl.doe.gov, djuna.gulliver@netl.doe.gov, christina.lopano@netl.doe.gov should have initial access.

4. Required Access Levels

If deploying only as a desktop application (compiled with PyInstaller), GCP access may not be required. Admins, Djuna and Rowan. Rowan (and Paige not NETL), developers. Christina, viewer.

Section 3: Application / Sandbox Use Case Description

5. Primary Purpose

DiGiTerra is a desktop application for data exploration, preprocessing, and machine learning model training. It provides a user-friendly interface for: - Data exploration and visualization (correlation matrices, PCA, statistical summaries) - Data preprocessing (missing value handling, scaling, feature selection) - Machine learning model training (regression, classification, clustering) - Model evaluation and visualization (performance metrics, SHAP summaries, feature importance) - Inference on new data using trained models

The application can be deployed in two ways: 1. **Desktop Application** (Primary): Standalone application compiled with PyInstaller 2. **Web Application** (Optional): Containerized application deployable to Kubernetes

6. Primary Tasks

- Hosting a NETL-approved executable that can be downloaded
- Upload and explore CSV datasets
- Perform statistical analysis and generate visualizations
- Preprocess data (handling missing values, scaling, feature selection)
- Train machine learning models (regression, classification, clustering)
- Evaluate model performance with cross-validation
- Generate predictions on new data
- Export results (Excel files, PDF visualizations)

7. Data Storage Needs

- **Desktop Application**: Stores user uploads and generated visualizations locally in `~/Library/Application Support/DiGiTerra/user_visualizations/`
- **Web Application**: Requires persistent storage for:
  - User-uploaded CSV files
  - Generated visualizations (PDFs, PNGs)
  - Model outputs (Excel files)
  - Estimated storage: 1-5 GB depending on usage (configurable via Helm values)

8. Computational Needs

- **CPU**: Standard CPU resources sufficient for most datasets
- **Memory**: Recommended 2-4 GB RAM for typical datasets
- **No GPU required**: All ML models use CPU-based scikit-learn implementations
- **Long-running jobs**: Model training can take several minutes for large datasets, but jobs are synchronous (user waits for completion)

9. GCP Use Case Type

If deploying only as a desktop application, I believe this just falls under hte Incubator Development section.

Section 4: General Hosting Requirements & Compliance

10. Source Code Repository Location

Current Private repository: `https://github.com/rowanterra/DiGiTerra`
Note: Several beta testers are listed as collaboraters at this time for access reasons.

11. Confirmation of EDX Team Access

Yes, Provided access to AVN email

12. TIC 3.0 Compliance Confirmation

We would like to know more about TIC compliance so that we can check this application complies.

As the desktop app runs locally and does not require network access beyond initial download, TIC 3.0 compliance may not be applicable for desktop deployment?

13. EDX Security (Cloud Armor) Compliance Confirmation

We have questions about Cloud Armor compliance. Is Cloud Armor compliance applicable for desktop deployment?

14. Public Facing Application Approval

Yes and No, the version approved by EDX will be an internal tool for data analysis and machine learning. Updated public facing versions may be available eslewhere, such as at Rowan's partner university or personal website.

Section 5: CI/CD Pipeline Best Practices

15. CI/CD Pipeline Usage

Currently, the application is built manually using PyInstaller. There is no CI/CD Pipeline.

16. Branch Protection Rules

Yes, but cannot be applied until public repository. ### 17. Pull Request Requirements Yes, or at least we are converting to this when deploying. ### 18. Commit Approvals Yes, 1.

Section 6: Application Containerization

19. Containerization Status

Yes. The application has Docker containerization assets available in `deploy/docker/`, but the primary deployment method is a standalone desktop application compiled with PyInstaller.

The desktop application (compiled macOS `.app`) does not require containerization as it runs natively. Containerization is available for web deployment scenarios.

Section 7: Docker Details

20. Dockerfile Location(s)

- `deploy/docker/Dockerfile`

21. Custom Build Arguments or Environment Variables

- `DIGITERRA_HOST`: Set to `0.0.0.0` (default in Dockerfile)
- `DIGITERRA_PORT`: Set to `5000` (default in Dockerfile)
- `PYTHONDONTWRITEBYTECODE=1`: Prevents Python from writing `.pyc` files
- `PYTHONUNBUFFERED=1`: Ensures Python output is unbuffered for proper logging

22. Multi-Stage Builds

No, Single-stage build using `python:3.11-slim` base image.

23. Base Image Selection

Base Image: `python:3.11-slim` - Lightweight Debian-based image - Includes Python 3.11 runtime - Sufficient for Flask application and scikit-learn dependencies - Reduces image size compared to full Python image

24. Exposed Ports

- **Port 5000**: HTTP port for Flask web application

25. Container Entrypoint and Command

- **ENTRYPOINT**: Not explicitly set (uses default)
- **CMD**: `["python", "app.py"]`

26. Graceful Shutdown (SIGTERM Handling)

Yes, Flask development server handles SIGTERM. The application can be enhanced with proper signal handling for production use (e.g., using gunicorn with proper worker management).

27. Image Size and Optimization

- **Base image**: `python:3.11-slim` (~45 MB)
- **Estimated final size**: ~400 MB (includes Python, Flask, scikit-learn, pandas, matplotlib, seaborn, SHAP, and other ML dependencies)
- **Optimization strategies**:
  - Uses `--no-cache-dir` for pip installs
  - Single-stage build to minimize layers
  - Runs as non-root user

28. Logging Configuration

Logging: Application logs to stdout/stderr (standard Python logging) **Log file location** (desktop app): `~/Library/Logs/DiGiTerra/digiterra.log` **Container logging**: All output goes to stdout/stderr for Kubernetes log aggregation.

29. Log File Details

- **Desktop application**: `~/Library/Logs/DiGiTerra/digiterra.log`
- **Containerized application**: Logs to stdout/stderr (no file-based logging in container)

30. Container User / Permissions

Yes, Application runs as non-root user: - User: `app` - Group: `app` - UID/GID: System-assigned (created with `adduser --system`)

31. Reason for Root Privileges

Not applicable.

Section 8: Helm Chart Availability

32. Helm Chart Availability

Yes, Helm chart is available at `deploy/helm/digiterra/`

Section 9: Helm Deployment Details

33. Helm Chart Location and Structure

- **Location**: `deploy/helm/digiterra/`
- **Structure**:
  - `Chart.yaml`: Chart metadata
  - `values.yaml`: Default configuration values
  - `templates/`:
    - `deployment.yaml`: Kubernetes Deployment
    - `service.yaml`: Kubernetes Service
    - `ingress.yaml`: Kubernetes Ingress (optional)
    - `pvc.yaml`: PersistentVolumeClaim for data storage
    - `_helpers.tpl`: Template helpers

34. Configurability via values.yaml

Yes, Chart is highly configurable via `values.yaml`: - `replicaCount`: Number of pod replicas - `image.repository` and `image.tag`: Container image lo-

cation - `service.port`: Service port (default: 5000) - `ingress.enabled`: Enable/disable ingress - `persistence.enabled`, `persistence.size`, `persistence.storageClassName`: Persistent storage configuration - `resources`: CPU and memory requests/limits - `env`: Environment variables (DIGITERRA_HOST, DIGITERRA_PORT)

35. Kubernetes Resource Definitions

- **Deployment**: Defines the application pods, replicas, container image, ports, environment variables, probes, and volume mounts
- **Service**: ClusterIP service exposing port 5000
- **Ingress**: Optional ingress resource (disabled by default)
- **PersistentVolumeClaim**: Optional PVC for persistent data storage (disabled by default)

36. CPU and Memory Requests / Limits

Currently not set in values.yaml - Should be configured based on workload: - **Recommended requests**: - CPU: 500m-1000m - Memory: 2Gi-4Gi - **Recommended limits**: - CPU: 2000m-4000m - Memory: 4Gi-8Gi These can be set in `values.yaml` under the `resources` section.

37. Health Checks (Liveness and Readiness Probes)

Yes, Both probes are configured in `deployment.yaml`: - **Liveness Probe**: - HTTP GET on path `/` port `http` - `initialDelaySeconds: 10` - `periodSeconds: 20` - **Readiness Probe**: - HTTP GET on path `/` port `http` - `initialDelaySeconds: 5` - `periodSeconds: 10`

38. Configuration Management

Configuration is managed via: - **Environment variables**: Set in `values.yaml` under `env` section - **ConfigMaps/Secrets**: Can be added to deployment template if needed - **Persistent storage**: Configured via `persistence` section in `values.yaml`

Section 10: Details for Apps Without a Helm Chart

39. CPU / RAM Utilization Estimates

Helm chart is available. See Section 9 for details.

If deploying without Helm: - **CPU**: 500m-2000m typical usage - **RAM**: 2-4 GB typical usage (can spike during model training with large datasets)

40. Persistent Storage Requirements

Helm chart includes PVC support. See Section 9.

If deploying without Helm: - **Storage needed**: 1-5 GB for user uploads and generated files - **Access mode**: ReadWriteOnce - **Mount path**: `/home/app/Library/Application Support/DiGiTerra` (or custom path)

41. GCS Fuse Mounts

No.

Section 11: Non-Containerized Application Details

42. Local Installation Guide

Desktop Application (macOS): 1. Download prebuilt `DiGiTerra.app` from releases 2. Drag `DiGiTerra.app` to `/Applications` 3. Double-click to launch

Development Setup:

```
pip install -r requirements.txt
python desktop_app.py  # For desktop window
# OR
python app.py  # For browser-based interface
```

43. Operating System Constraints

- **Desktop Application**: macOS (currently), can be extended to Windows and Linux
- **Web Application**: Linux (containerized)
- **Python Version**: 3.11+
- **Cross OS Guides** are available per the README.md.

44. Required System & Application Packages

Python Dependencies (see `requirements.txt`): - Flask (web framework) - pandas, numpy (data processing) - scikit-learn (machine learning) - matplotlib, seaborn (visualization) - SHAP (model explainability) - xlsxwriter, openpyxl (Excel export) - pywebview (desktop app wrapper)

System Requirements: - macOS: No additional system packages required for compiled app - Development: Python 3.11+, pip

45. User / Group Application Permissions

- **Desktop Application**: Runs with current user's permissions
- **Container**: Runs as non-root user `app:app`

- **File Permissions**: Application creates directories in user's home directory (`~/Library/Application Support/DiGiTerra/`)

46. Environment Variables

- `DIGITERRA_PORT`: Port number for Flask server (default: random free port)
- `DIGITERRA_DEBUG`: Enable debug logging (set to `1` to enable)
- `DIGITERRA_HOST`: Host address (default: `127.0.0.1` for desktop, `0.0.0.0` for container)
- `DIGITERRA_BASE_DIR`: Base directory for application (optional)
- `DIGITERRA_OUTPUT_DIR`: Output directory for visualizations (default: `~/Library/Application Support/DiGiTerra/user_visualizations/`)

Section 12: Data, Secrets, and Scalability

47. Estimated Data Size and Growth Rate

- **Initial data**: Minimal (application code and dependencies)
- **User-generated data**:
  - Uploaded CSV files: Typically 1-100 MB per file
  - Generated visualizations: 1-50 MB per analysis session
  - Model outputs: 1-10 MB per model
- **Growth rate**: Low to moderate (depends on number of users and frequency of use)
- **Estimated total**: 1-5 GB per active user per year

48. Multi-Region Requirements

No, Single region deployment is sufficient. Application does not require multi-region deployment.

49. Multi-Region Details

Not applicable.

50. Data Lifecycle Policy

- **User uploads**: Can be deleted after processing (temporary)
- **Generated visualizations**: Retained for user download, can be cleaned up after a retention period
- **Model outputs**: Retained for user download, can be cleaned up after retention period

51. Application Secrets

Application does not use secrets or API keys. The application does not require authentication, API keys, or external service credentials. All functionality is self-contained.

52. Autoscaling Requirements

Not currently configured - Application can be configured for V/HPA based on CPU/memory usage if needed.

53. Replicas and Scalability Details

If downloaded as an application that is fully compiled, this should not have to be considered. - **Initial replicas**: 1 (configurable via `replicaCount` in Helm values) - **Scaling strategy**: - Stateless application (no session state), can scale horizontally - Each pod handles requests independently - Consider sticky sessions if user-specific data caching is implemented - **Resource constraints**: Model training is CPU/memory intensive; ensure adequate resources per pod

---

Section 13: Support and Operations & Maintenance

54. Process for Security Patches

Process: 1. **Monitoring**: - Use `pip-audit` or GitHub Dependabot to monitor Python dependency vulnerabilities - Subscribe to security advisories for Flask, scikit-learn, pandas, and other critical dependencies - Monitor CVE databases for base Docker image vulnerabilities

2. **Assessment**:
   - Evaluate severity and impact of security patches
   - Test patches in development/staging environment before production
   - Review changelogs for breaking changes
3. **Deployment**:
   - Apply critical security patches immediately
   - Apply non-critical patches during scheduled maintenance windows. Each month during the first twelve months, and then to quarterly and biannual or as needed.
   - Maintain rollback capability via Helm chart versioning

55. Process for Application Hot Fixes

Process: 1. **Critical Hot Fixes** (security, data loss, service outage): - Immediate assessment and development - Deploy to staging environment for validation - Post-deployment monitoring and verification

2. **Non-Critical Hot Fixes**:

    - Deploy to staging environment first
    - Full testing in staging before production
    - Deploy during scheduled maintenance windows when possible
3. **Deployment Strategy**:
    - Maintain rollback capability via Helm chart versioning
    - Document all hot fixes in change log
    - Communicate changes to users if needed
4. **Post-Deployment**:
    - Monitor application logs and metrics
    - Verify fix resolves the issue
    - Update documentation as needed

56. Process for Application Version Upgrades

Has not been fully applied yet.

Process: 1. **Versioning**: - Use semantic versioning (e.g., v1.0.0, v1.1.0, v2.0.0) - Follow semantic versioning rules: MAJOR.MINOR.PATCH - Tag releases in Git repository - Use Helm chart versioning to track application versions

2. **Pre-Upgrade**:
    - Test upgrades in development/staging environment first
    - Document breaking changes and migration requirements
    - Provide migration guides for major version upgrades
    - Communicate upgrade schedule to users
3. **Upgrade Deployment**:
    - Deploy during scheduled maintenance windows
    - Maintain rollback capability
    - Monitor application health during and after upgrade
4. **Post-Upgrade**:
    - Verify all functionality works as expected
    - Monitor logs and metrics for issues
    - Update documentation

57. Reliability and Backup Requirements

- **Backup strategy**:
    - Application code: Version controlled in Git repository
    - User data: If persistent storage is enabled, implement regular backups of PVC data
    - Configuration: Store in version control or ConfigMaps
- **Reliability**:
    - Health checks ensure pod restarts on failure
    - Multiple replicas provide high availability
    - Persistent storage ensures data survives pod restarts

Section 14: Standalone Incubator Sandbox Requirements

58. Required GCP Technologies

[To be filled in by applicant - list required GCP services]

Likely requirements (if deploying web version): - Google Kubernetes Engine (GKE) or Cloud Run - Cloud Storage (GCS) - optional, for data storage - Container Registry (GCR) or Artifact Registry - for Docker images - Cloud Build - optional, for CI/CD

Note: Desktop application does not require GCP services.

59. User Permissions Within Sandbox

- Pull access to container registry
- Download application to device.

60. Sandbox Backup Requirements

May not be applicable if downloadable app. - Git repository serves as backup and updates for application code

---

Notes and Clarifications

Desktop vs. Web Deployment

This application has two deployment modes:

1. **Desktop Application (Primary)**:
   - Compiled macOS/Windows/Linux application using PyInstaller
   - Runs locally on user's machine
   - No cloud infrastructure required
   - Many questions in this form (Docker, Helm, GCP, etc.) do not apply
   - **Status**: Production-ready for desktop use
   - **Security**: Appropriate security measures for single-user desktop deployment (see `SECURITY_REVIEW.md`)
2. **Web Application (Optional)**:
   - Containerized Flask application
   - Deployable to Kubernetes/GCP
   - All Docker/Helm/Kubernetes questions apply to this deployment mode
   - **Status**: Additional security hardening required for multi-user web deployment
   - **Security**: See `SECURITY_REVIEW.md` for required security measures (CSRF, authentication, rate limiting, etc.)

Security Documentation

- `SECURITY_REVIEW.md`: Comprehensive security assessment, current security measures, and recommendations for web deployment
- `ISSUES_FOUND.md`: Historical security fixes and remaining code quality improvements
- `HANDOFF.md`: Developer notes including security considerations
- `DEBUG_REPORT.md`: Current status and verification results